

1. How did implementing debouncing improve the reliability of switch inputs?

Mechanical switches like SW1 and SW2 do not transition cleanly from open to closed. When pressed, the contacts **bounce**, generating multiple rapid on/off signals. Without debouncing, the program could **misread a single press as multiple presses**, causing the counter to increment or decrement unexpectedly. By adding a **small delay and verifying the switch state**, the debouncing ensures that each press is **counted only once**, making the counter output stable and predictable.

2. Why is it necessary to configure pull-up resistors for SW1 and SW2?

Pull-up resistors ensure that the input pins (PF4 and PF0) have a defined voltage when the switches are **not pressed**. Without them, the pins would **float**, potentially reading as random high or low signals, which could cause the counter to behave erratically.

The **internal pull-up resistors** connect the pins to **logic high (3.3V)** when unpressed. Pressing the switch connects the pin to ground, generating a reliable **logic low** signal.

3. How does using modular functions improve the readability and maintainability of your code?

Functions like `Set_Up()`, `Set_Down()`, `Display_Number()`, `PortB_Init()`, and `PortF_Init()` **encapsulate specific tasks**.

- Improves **readability**: `main()` becomes clean and easy to understand because the logic is abstracted.
 - Improves **maintainability**: Changing the behavior of incrementing, decrementing, or display logic only requires updating the corresponding function without affecting the rest of the code.
 - Encourages **code reuse** and prevents repeated code blocks.
-

4. Suppose you needed to control external LEDs instead of the onboard ones. What changes would be required in the circuit and code?

- **Circuit changes**: Connect the external LEDs to the appropriate GPIO pins, using **current-limiting resistors** to prevent damage.
- **Code changes**: Update the `Display_Number()` function or the GPIO initialization to point to the pins connected to the external LEDs instead of

PB0-PB3. Ensure GPIOx->DIR and GPIOx->DEN are set correctly for the new pins.

- Optionally, modify the **mapping** if the LEDs represent different bits or patterns.

5. If the counter needed to display numbers beyond 0-9, what modifications would you make?

- Update the counter variable to handle a larger range (e.g., uint8_t or uint16_t).
- Change Display_Number() to **drive multiple digits**, either by:
 - Using **multiple 7-segment displays** with digit multiplexing, or
 - Using a **BCD to 7-segment decoder** like 74LS47 for each digit.
- Modify the increment and decrement logic to wrap appropriately for the new range (e.g., 0-99 for two digits).
- Ensure timing and display update logic can handle **multi-digit output**.